Mitali Bhatia
D15C 16
MLDL

Experiment No 04

Aim: Implementation of K-Nearest Neighbors (KNN) Algorithm and evaluate model performance

1. Dataset Source

The dataset used for this experiment is the **Mall Customers Dataset**, which is publicly available on Kaggle.

Dataset Link:
https://www.kaggle.com/datasets/abdallahwagih/mall-customers-segmentation

This dataset is commonly used for customer analysis and segmentation tasks and is suitable for demonstrating classification algorithms like K-Nearest Neighbors.

---

2. Dataset Description

The Mall Customers dataset contains information about customers visiting a shopping mall. The objective of this experiment is to classify customers into different **spending categories** based on their demographic and purchasing behavior.

Dataset Characteristics

- Number of instances: 200 customers
- Type: Structured dataset
- Domain: Marketing / Customer Analytics
- Problem Type: Multi-class classification

Features Used

- **Genre** – Gender of the customer (Male/Female)
- **Age** – Age of the customer
- **Annual Income (k$)** – Annual income in thousand dollars
- **Spending Score (1–100)** – Score assigned based on customer spending behavior

Target Variable

- **Spending_Category** (derived feature)
  - 0 → Low spending
  - 1 → Medium spending
  - 2 → High spending

The target variable is created by grouping the spending score into categories to make the dataset suitable for classification.

---

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a **non-parametric and instance-based learning algorithm**. It does not build an explicit model but makes predictions based on similarity between data points.

Distance Calculation

The most commonly used distance metric in KNN is **Euclidean Distance**:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Where:

- x = test data point
- y = training data point
- n = number of features

Classification Rule

1. Calculate the distance between the test point and all training points
2. Select the **K nearest neighbors**
3. Assign the class based on **majority voting** among the neighbors

---

4. Algorithm Limitations

- Computationally expensive for large datasets
- Performance depends heavily on the choice of K value
- Sensitive to irrelevant features and noise
- Requires feature scaling for accurate distance measurement
- Suffers from the curse of dimensionality in high-dimensional data

---

5. Methodology / Workflow

Step-by-Step Procedure

1. Load the dataset using Pandas
2. Encode categorical features such as gender
3. Create spending categories from spending score
4. Select input features and target variable
5. Apply feature scaling using standardization
6. Split the dataset into training (80%) and testing (20%) sets
7. Train the KNN classifier
8. Predict spending categories on test data
9. Evaluate model performance using classification metrics

Workflow Diagram (Textual Representation)

Dataset → Preprocessing → Feature Scaling → Train-Test Split
→ KNN Model Training → Prediction
→ Performance Evaluation

---

6. Performance Analysis

The model performance is evaluated using the following metrics:

- **Accuracy:** Measures the proportion of correct predictions
- **Confusion Matrix:** Shows correct and incorrect classifications
- **Classification Report:** Includes precision, recall, and F1-score

Observations

- KNN provides good classification accuracy for customer spending behavior
- Feature scaling significantly improves performance
- The model effectively distinguishes between low, medium, and high spenders

---

7. Hyperparameter Tuning

Tuned Hyperparameter

- **K (Number of Neighbors)**

Tuning Approach

- Tested different values of K ranging from 1 to 20
- Calculated accuracy for each K value
- Selected the K value with the highest accuracy

Impact of Tuning

- Small K values caused overfitting
- Large K values caused underfitting
- Optimal K value improved generalization and reduced classification error

---

CODE:

```
# ================================
# IMPORT LIBRARIES
# ================================
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report


# ================================
# LOAD DATASET
# ================================
df = pd.read_csv("Mall_Customers.csv")

print(df.head())
```

```python
print(df.columns)
print(df.shape)


# ================================
# DATA PREPROCESSING
# ================================
# Encode Genre column
df['Genre'] = df['Genre'].map({'Male': 0, 'Female': 1})


# ================================
# CREATE TARGET (FOR CLASSIFICATION)
# ================================
df['Spending_Category'] = pd.cut(
    df['Spending Score (1-100)'],
    bins=[0, 40, 70, 100],
    labels=[0, 1, 2]
)


# ================================
# FEATURES & TARGET
# ================================
X = df[['Genre', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
y = df['Spending_Category']


# ================================
# FEATURE SCALING
# ================================
scaler = StandardScaler()
X = scaler.fit_transform(X)


# ================================
# TRAIN-TEST SPLIT
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)


# ================================
# KNN CLASSIFIER
# ================================
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)


# ================================
# PERFORMANCE EVALUATION
# ================================
print("\n--- KNN Classifier ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))


# ================================
# HYPERPARAMETER TUNING (K VALUE)
# ================================
k_values = range(1, 21)
accuracy_list = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred_k = model.predict(X_test)
    accuracy_list.append(accuracy_score(y_test, pred_k))


plt.plot(k_values, accuracy_list, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy vs K Value")
plt.show()
```

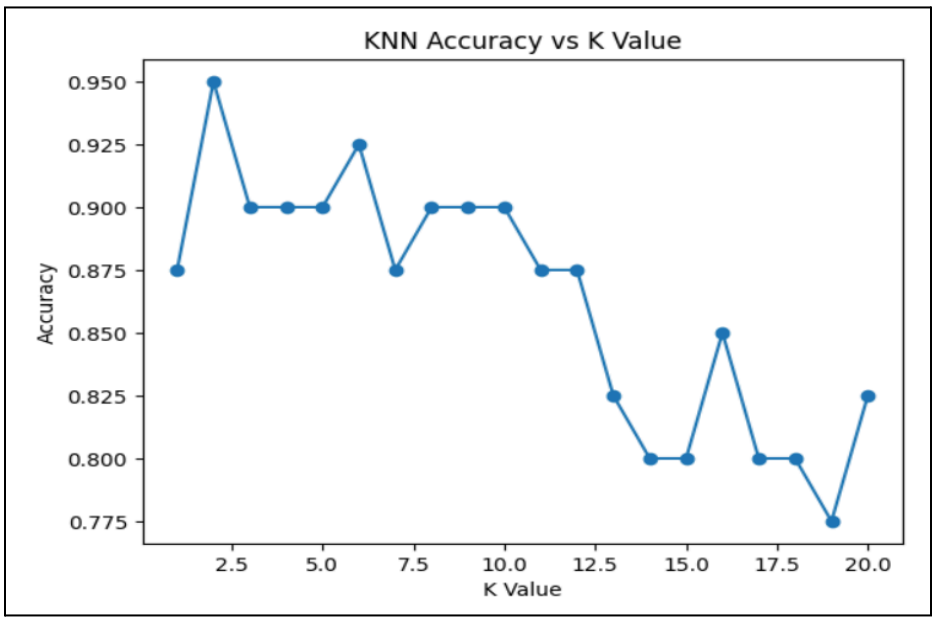
OUTPUT:

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
Index(['CustomerID', 'Genre', 'Age', 'Annual Income (k$)',
       'Spending Score (1-100)'],
      dtype='object')
(200, 5)

--- KNN Classifier ---
Accuracy: 0.9
Confusion Matrix:
 [[13  3  0]
 [ 0 17  1]
 [ 0  0  6]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.81      0.90        16
           1       0.85      0.94      0.89        18
           2       0.86      1.00      0.92         6

    accuracy                           0.90        40
   macro avg       0.90      0.92      0.90        40
weighted avg       0.91      0.90      0.90        40
```



Conclusion

This experiment successfully demonstrated the application of the K-Nearest Neighbors algorithm on a real-world mall customer dataset. Proper preprocessing, feature scaling, and hyperparameter tuning significantly improved model performance. KNN proved to be a simple yet effective algorithm for customer classification tasks.